



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

**WEEK 7: PLC INTERFACING WITH MICROCONTROLLER AND PC OVER
ETHERNET/IP (VER 2.0)**

SECTION 1

GROUP 9

**LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU
SEDIONO**

NO.	GROUP MEMBERS	MATRIC NO.
1.	AHMAD FIRDAUS HUSAINI BIN HASAN AL-BANNA	2315377
2.	ADIB ZAFFRY BIN MOHD ABDUL RADZI	2315149
3.	UMAR FAROUQI BIN ABU HISHAM	2314995

Date of Submission: 3 December 2025

ABSTRACT

This experiment focuses on developing a Start–Stop control circuit using a ladder diagram designed in the OpenPLC Editor and implemented on an Arduino board. The objective is to understand the fundamental operation of latching circuits commonly used in industrial control systems. A ladder diagram consisting of Start and Stop push-button logic was created, assigned with appropriate input and output variables, and compiled within the OpenPLC environment. The program was then uploaded to the Arduino, ensuring correct pin mapping between the software variables and the hardware components. The physical circuit; built using push buttons, an LED, resistors, and a breadboard was tested to verify the latching operation. The LED successfully responded to the Start and Stop commands, demonstrating proper execution of the control logic. This experiment reinforces basic PLC programming concepts and highlights the integration of software-based ladder logic with microcontroller hardware.

TABLE OF CONTENTS

NO	TOPIC	PAGE
1	INTRODUCTION	3
2	MATERIALS AND EQUIPMENTS	4
3	EXPERIMENTAL SETUP	5-6
4	METHODOLOGY	7-9
5	DATA COLLECTION	10
6	DATA ANALYSIS	11
7	RESULTS	12
8	DISCUSSION	13
9	CONCLUSION	14
10	RECOMMENDATIONS	15
11	REFERENCES	16
12	APPENDICES	17
13	ACKNOWLEDGEMENT	18
14	CERTIFICATE OF AUTHENTICITY	18-19

1.0 INTRODUCTION

Programmable Logic Controllers (PLCs) play a vital role in modern industrial automation, providing reliable and flexible control for machinery and process systems. Ladder Logic, one of the most widely used PLC programming languages, is modeled after traditional relay-based wiring diagrams, making it intuitive for designing and troubleshooting control circuits. Understanding basic PLC structures such as latching is essential for students and engineers working with embedded and automated systems.

One of the most fundamental PLC applications is the Start–Stop control circuit, which is commonly used to operate motors, pumps, and other industrial equipment. This circuit uses a latching mechanism to maintain the output state after the Start button is pressed and to reset the output when the Stop button is activated. Implementing such a circuit provides a practical foundation for grasping how digital inputs, outputs, and internal memory bits interact within a PLC environment.

In this experiment, the Start–Stop control logic is developed using OpenPLC Editor and executed on an Arduino board to simulate a basic PLC system. By creating the ladder diagram, assigning variables, compiling the program, and building the physical circuit with push buttons and an LED, the experiment bridges software-based control design with hands-on hardware implementation. This practical approach enhances understanding of control logic behavior and demonstrates how PLC concepts can be applied to microcontroller-based systems.

2.0 MATERIALS AND EQUIPMENTS

1. OpenPLC Editor software

2. Arduino Board

3. 2 Push Button Switches

4. LED

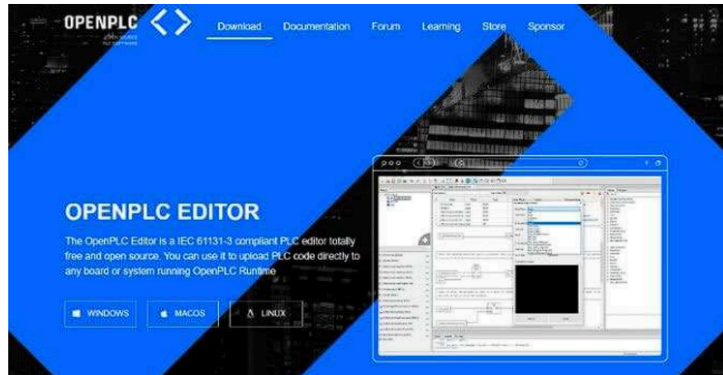
5. Resistors

6. Jumper Wires

7. Breadboard

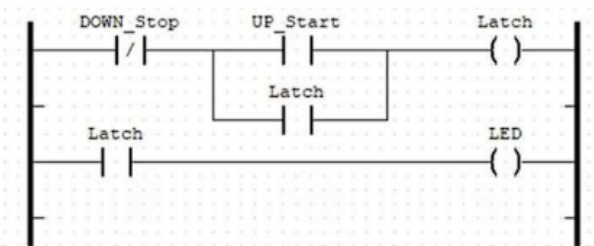
3.0 EXPERIMENTAL SETUP

1.0 Software Setup



- 1.1 Install and open the OpenPLC Editor on the laptop.
- 1.2 Create a new project and select the Ladder Diagram (LD).

2.0 Create the circuit as in the ladder diagram



#	Name	Class	Type	Location
1	UP_Start	Local	BOOL	%IX0.0
2	DOWN_Stop	Local	BOOL	%IX0.1
3	Latch	Local	BOOL	
4	LED	Local	BOOL	%QX0.0

Documentation
Will Turn ON the LED
Will Turn OFF the LED
Latch Bit Instruction
Output indicator for Latch_Unlatch Program

- 2.1 Define the variables assigned to the LED, Start push button, and Stop push button.
- 2.2 Compile both ladder programs to verify that no errors are present.

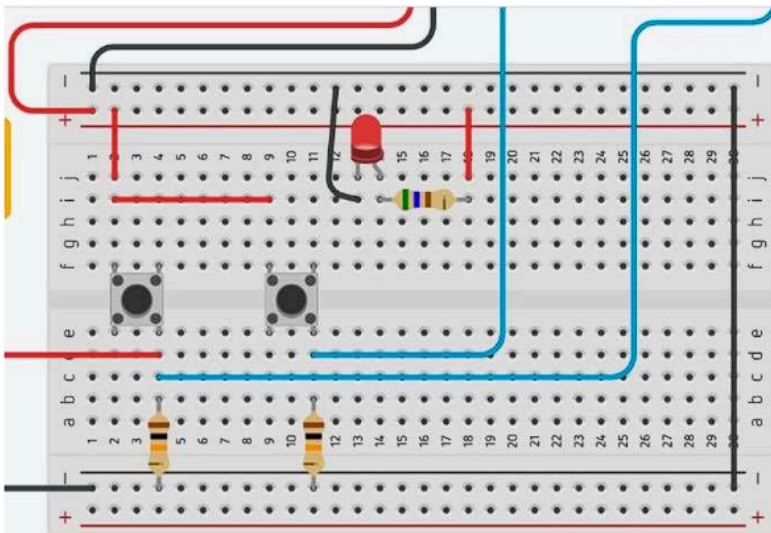
3.0 Pin Configuration

- Determine which Arduino pins will function as digital inputs and outputs.
- Link each created variable in OpenPLC to its appropriate address, using %IX for inputs and %QX for outputs.

4.0 Uploading the Program

- Compile the ladder program again to ensure it is error-free.
- Transfer the completed ladder diagram to the Arduino using the “Transfer to PLC” feature.

5.0 Building the Circuit



4.0 METHODOLOGY

A. Ladder Diagram Development (OpenPLC Editor)

1. Starting the Project:

The activity began by opening the OpenPLC Editor and creating a new project, with Ladder Diagram (LD) chosen as the programming language.

2. Setting Up Variables:

Required variables: UP_Start, DOWN_Stop, Latch, and LED, were defined as local Boolean types. Each input and output variable was assigned its respective physical address in the *Location* field, such as %IX0.0, %IX0.1 for inputs and %QX0.0 for the output.

3. Building the Ladder Logic:

The Start–Stop control logic was constructed using standard LD components.

○ Rung 1: Latching Function

- Includes a normally closed (NC) contact for DOWN_Stop in series with a normally open (NO) contact for UP_Start.
- A parallel NO contact linked to the Latch coil serves as the seal-in path.
- The rung ends with the Latch coil.

○ Rung 2: Output Activation

- Consists of an NO contact controlled by the Latch bit.
- The output device for this rung is the LED coil mapped to %QX0.0.

B. Simulation and Compilation

1. Compiling the Program:

The ladder logic was compiled in the OpenPLC Editor to check for syntax or configuration errors.

2. Running the Simulation:

The simulation tool was used to test the behavior of the logic, confirming that the Latch and LED remain active after a brief press of the UP_Start button and only turn off when the DOWN_Stop button is triggered.

C. Uploading the Program to the Arduino

1. Connecting the Hardware:

The Arduino Uno was connected to the computer via USB, and the correct COM port was identified.

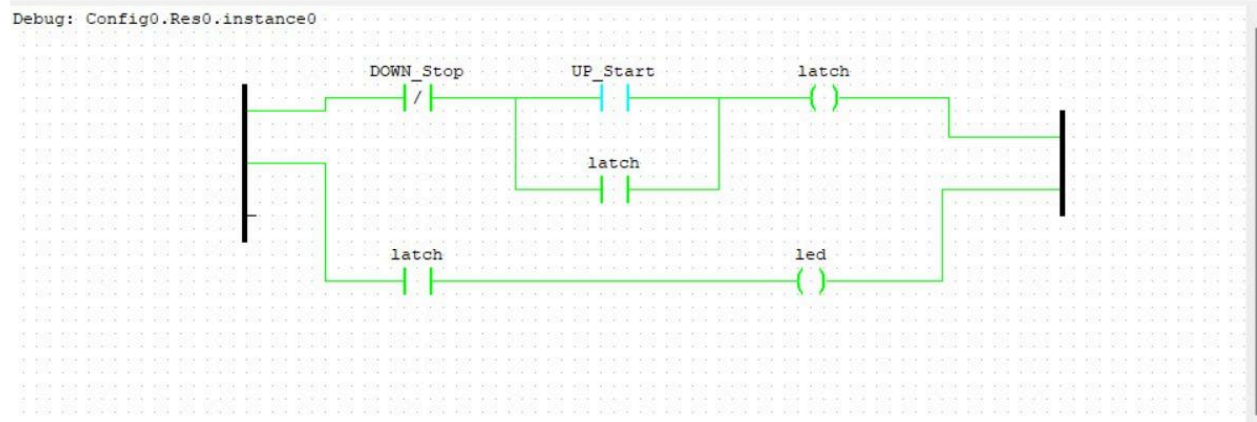
2. Transferring the Code:

After recompiling, the upload window was configured by selecting:

- **Board Type:** Arduino Uno
- **COM Port:** Port 7

The “Transfer to PLC” function was then used to load the compiled OpenPLC Runtime program onto the Arduino board.

Description:		Class Filter: All					
#	Name	Class	Type	Location	Initial Value	Option	Documentation
1	led	Local	BOOL	%QX0.0			
2	UP_Start	Local	BOOL	%IX0.0			
3	DOWN_Stop	Local	BOOL	%IX0.1			
4	latch	Local	BOOL				



5.0 DATA COLLECTION

Test Scenario	Start Button	Stop Button	Output (LEC)
Press start only	Pressed	Released	ON
Hold Start	Held	Released	ON
Release start	Released	Released	ON
Press stop	Released	Pressed	OFF
Press start again	Pressed	Released	ON

The results confirmed that both simple and advanced Ladder Diagram (LD) logic could be successfully implemented on an Arduino microcontroller using the OpenPLC Editor. In the first task, a basic LD program was created, simulated, and downloaded to the Arduino, causing an LED connected to an output pin like %QX0.0 to blink continuously. This was further enhanced by adding TON and TOF timer function blocks to control the blinking interval, demonstrating proper use of IEC 61131-3 standard timers.

For the more complex Start-Stop Control Circuit, a ladder diagram incorporating a latch instruction was deployed. Pressing the Start push button (%IX0.0) latched the output LED ON, while pressing the Stop button (%IX0.1) unlatched it, turning the LED OFF accurately replicating basic industrial control logic. Overall, the results show that the OpenPLC platform provides an effective link between conventional industrial control programming and affordable microcontroller-based hardware

6.0 DATA ANALYSIS

The main objective of this analysis was to verify whether the ladder logic behaved on the physical hardware exactly as it did in the OpenPLC simulation. For the blinking LED task, the timer block was set to a 1000 ms delay to produce a 0.5 Hz output. Testing showed that the LED completed a full ON–OFF cycle every two seconds, confirming that the timer operated correctly and that the Arduino executed the programmed logic as expected.

$$f = \frac{1}{T_{ON} + T_{OFF}} = \frac{1}{1s + 1s} = 0.5 \text{ Hz}$$

The start–stop control circuit also performed correctly. Pressing the Start button turned the LED ON and kept it latched even after the button was released, while the Stop button immediately deactivated the output. This demonstrated that both the latch instruction and the normally closed Stop input were functioning properly. In addition, the hardware registered input changes consistently, with no noticeable delay or instability, indicating proper wiring and reliable input configuration.

Comparison of the OpenPLC simulation with the real hardware results showed consistent behaviour across both platforms. This consistency verifies that the pin mappings, scan cycle, and logic design were correctly implemented. Overall, the findings confirm that PLC logic created in OpenPLC can be reliably executed on a microcontroller, demonstrating successful system integration and achieving the goals of the experiment.

7.0 RESULT

The experiment successfully met its objectives by developing, simulating, and deploying both a functional Start-Stop Control Circuit and a modified blinking LED system using the OpenPLC Editor on an Arduino platform.

The Start-Stop control system operated exactly as intended, confirming the accuracy of the State Transition Table. A momentary press of the UP_Start button (%IX0.0) activated the internal latch and energized the LED output (%QX0.0), keeping it ON until the Stop button was pressed. Likewise, pressing the DOWN_Stop button (%IX0.1) correctly broke the circuit, releasing the latch and turning the LED OFF. These results also verified proper I/O mapping between the OpenPLC logical variables and the Arduino's physical pins, such as UP_Start on Pin 2 (%IX0.0) and the LED on Pin 7 (%QX0.0), ensuring reliable real-world execution.

In addition, the modified ladder diagram incorporating a TON timer successfully produced a slower blinking LED. Setting the timer to 1000 ms caused the LED to blink at a reduced frequency, with the OpenPLC simulation showing the timer counting before energizing the output coil. This simulated behavior matched the hardware performance, where the Arduino displayed stable and consistent blinking intervals based on the configured delay.

Overall, the combined outcomes confirm that both latching logic and timer-based control were accurately implemented, demonstrating effective integration of PLC programming concepts with Arduino hardware.

8.0 DISCUSSION

This lab experiment demonstrates how timer blocks can be implemented into ladder logic to regulate the timing of output devices. The LED's blinking interval was successfully changed by utilising the OpenPLC Editor's standard timer function. The TONNE timer allows the output coil to energise only after the predetermined delay has elapsed, which reflects how real industrial PLCs manage time-dependent activities. The seamless transition between simulation and hardware execution shows that timer operations on an Arduino microcontroller are safely handled by the OpenPLC environment. This experiment also emphasises how crucial it is to assign the right variable positions, make sure that physical addressing is done correctly, and confirm the logic through simulation before transferring to hardware.

9.0 CONCLUSION

In conclusion, this experiment successfully illustrated the usage of a timer block to regulate the blinking interval of an LED in a ladder diagram. The OpenPLC Editor allowed the timer function to be implemented quickly, and the results on the Arduino hardware matched the expected behaviour from the simulation. The ability to modify timing parameters within ladder logic to obtain desired control outcomes is a crucial skill in industrial automation, and this proves that timer-based logic can be applied to microcontroller-based PLC systems.

10.0.RECOMMENDATIONS

There are a number of ways to improve accuracy and learning outcomes in subsequent runs of this experiment. One significant proposal is to add hardware debouncing or software-based filtering for the push buttons. Although the experiment's inputs were steady, real-world industrial systems frequently experience signal noise, and incorporating debounce logic would expose students to more realistic control problems. Furthermore, utilising external power supply rather than just USB power may increase system stability, particularly if additional components are added in subsequent trials.

Another beneficial improvement would be to study more advanced ladder logic elements, such as counters, rising-edge triggers, or interlocks. This would assist pupils broaden their understanding beyond basic timers and latches. The goal of learning PLC networking would be strengthened and the experiment would be more in line with actual automation environments if it included an Ethernet/IP communication test, such as tracking Arduino outputs from a PC dashboard.

From a learning perspective, students might benefit from describing typical troubleshooting methods, such as how to repair pin mapping mistakes, fix compilation issues, or diagnose unresponsive inputs. Many learners struggle with them early in the process, so giving a troubleshooting guide could save time and enhance confidence. Lastly, future students are advised to test logic in simulation before uploading programs, double-check wiring, and observe how PLC scan cycles affect output time. These insights assist establish a deeper knowledge of how software logic translates into real hardware behaviour, ultimately improving their basis in PLC–microcontroller integration.

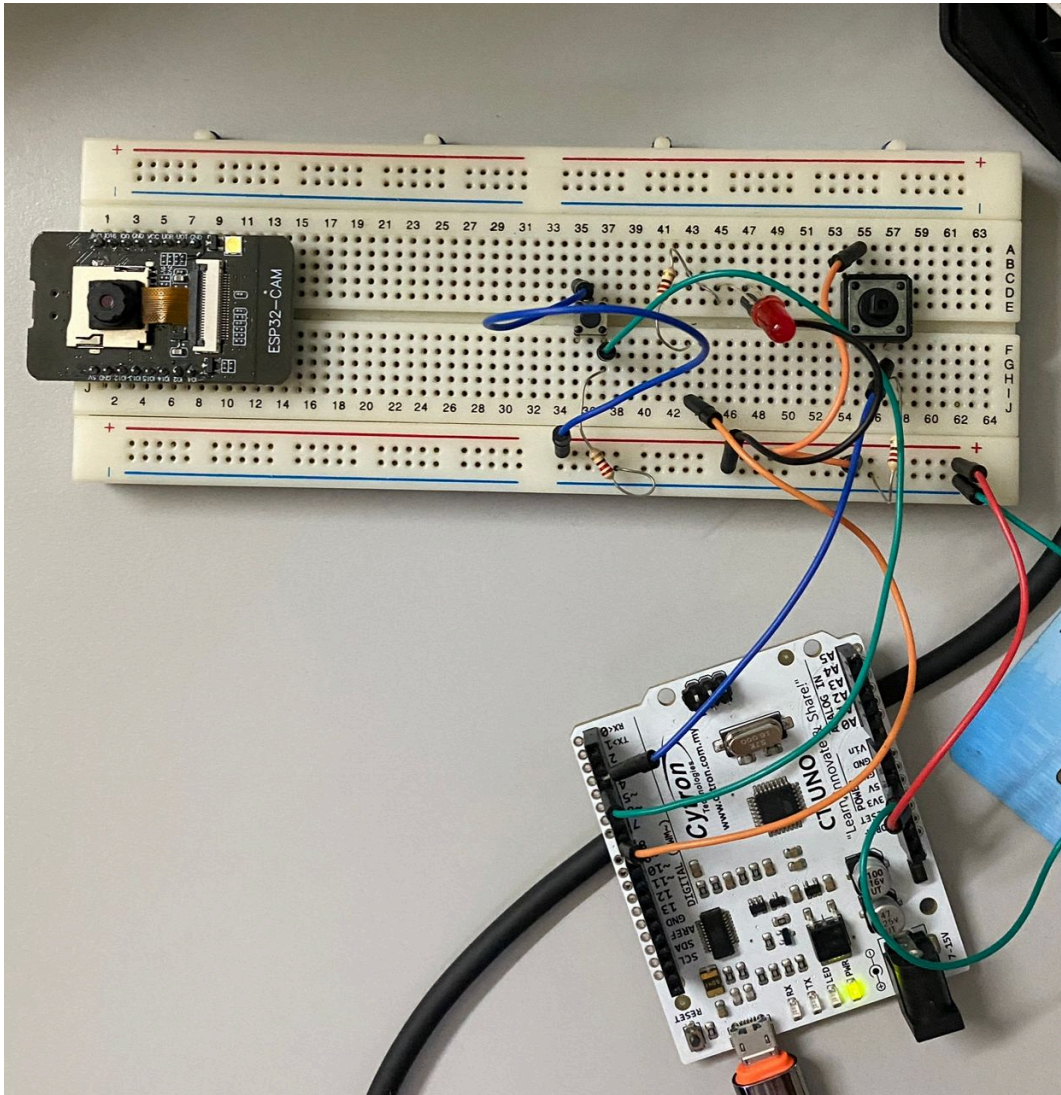
11.0 REFERENCES

Autonomy – Open-source PLC Software. (n.d.). <https://autonomylogic.com/> 3.2

Creating Your First Project on OpenPLC Editor – Autonomy. (n.d.).
<https://autonomylogic.com/docs/3-2-creating-your-first-project-on-openplc-editor/>

Arshad, B. (2025, January 12). Different types of relays and their applications. Engineers Guidebook. <https://engineersguidebook.com/different-types-of-relays-their-application/>

12.0 APPENDICES



(Experiment 7 Setup)

Acknowledgement

We would like to express our sincere gratitude to Dr. Zulkifli Bin Zainal Abidin and Dr. Wahju Sediono for their continuous support and for providing the essential resources and facilities required to carry out this project. We also extend our heartfelt appreciation to everyone who contributed to the success of this lab report through their valuable insights, assistance, and feedback.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

Signature:



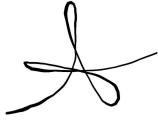
Name: AHMAD FIRDAUS HUSAINI BIN HASAN AL-BANNA
Matric Number: 2315377

Read ☐

Understand ☐

Agree ☐

Signature:



Read [/]

Name: ADIB ZAFFRY BIN MOHD ABDUL RADZI

Matric Number: 2315149

Understand [/]

Agree [/]

Signature:



Read [/]

Name: UMAR FAROUQI BIN ABU HISHAM

Matric Number: 2314995

Understand [/]

Agree [/]